

uvgComm: Open Software for Low-Latency Multi-party Video Communication

Joni Räsänen,
Ultra Video Group,
Tampere University,
joni.rasanen@tuni.fi

Heikki Tampio,
Ultra Video Group,
Tampere University,
heikki.tampio@tuni.fi

Alexandre Mercat,
Ultra Video Group,
Tampere University,
alexandre.mercat@tuni.fi

Jarno Vanne,
Ultra Video Group,
Tampere University,
jarno.vanne@tuni.fi

Abstract

Effective and smooth video communication has become an integral tool for interaction in our modern society. The well-known WebRTC project is optimized for server-based communication architectures with less stringent latency requirements. This paper introduces an open-source, peer-to-peer (P2P) mesh-based application called uvgComm 1.0 for low-latency multi-party video communication. Our experimental results show that uvgComm attains 25% lower latency than server-based solutions at 219 ms. In addition, it offers advanced privacy protection mechanisms with end-to-end encryption and P2P mesh topology that delivers media without any media servers. The ambitious goal of uvgComm is to take P2P mesh-based conferencing architecture to the next level and make this architecture a serious competitor to server-based communication architectures.

CCS Concepts

- Information systems → Multimedia information systems
- Software and its engineering → Peer-to-peer architectures
- Software and its engineering → Communications management

Keywords

Open source, real-time communications (RTC), multi-party video conferencing, Real-time Transport Protocol (RTP), peer-to-peer (P2P) mesh

ACM Reference format:

Joni Räsänen, Heikki Tampio, Alexandre Mercat and Jarno Vanne. 2024. uvgComm: Open Software for Low-Latency Multi-party Video Communication. In *Proceedings of ACM Multimedia 2024 conference (MM'24)*, Oct 28 – Nov 1, Melbourne, VIC, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3664647.3685514>



This work is licensed under a Creative Commons Attribution International 4.0 License.

MM '24, October 28–November 1, 2024, Melbourne, VIC, Australia.
© 2024 Copyright is held by the owner/author(s).
ACM ISBN 979-8-4007-0686-8/24/10.
<https://doi.org/10.1145/3664647.3685514>

1 Introduction

As organizations strive for productivity with a low carbon footprint, the video communication *quality of experience* (QoE) indicators, such as low latency that does not allow for a possibility of significant overlap between participants [1], have become crucial factors in competing with business travel. According to ITU-T recommendation [2], latency should be kept below 200 ms to ensure high user satisfaction and 400 ms to prevent significant user dissatisfaction. In a typical video communication system, the limited budget for latency must be distributed across several stages of the end-to-end video pipeline, including the video capture, encoding, delivery, decoding, and rendering. Especially, delivery latencies can amount to hundreds of milliseconds, which can significantly weaken the user QoE.

Existing video conferencing systems are typically based on the following three network topologies called *selective forwarding unit* (SFU), *multi-point control unit* (MCU), and *peer-to-peer* (P2P) mesh. In an SFU architecture, data is routed to a central server before reaching its destination, which inherently increases latency. MCU-based systems transcode the video at the MCU which increases the latency further. P2P mesh network typically provides the lowest latency and enhanced privacy, as it uses the most direct route between peers. Certain types of *network address translations* (NAT) can prevent the formation of P2P connections, but this limitation can be overcome by adopting IPv6.

Currently, there are two open-source solutions for P2P mesh-based video communication: Peer Calls [3] and Mooz [4]. They are both built on WebRTC [5] framework. Despite its widespread use, WebRTC has several limitations that make it less ideal for P2P mesh networks. First, it does not support *Zimmermann Real-time Transport Protocol* (ZRTP) [6], which is a key exchange protocol for secure media pathways that is not reliant on public key infrastructure. Secondly, WebRTC architecture needs a separate encoder for each media flow, increasing complexity and resource usage. Lastly, WebRTC uses *Google Congestion Control* (GCC) [7] that does not support coupling, a process of combining the *quality of service* (QoS) information from multiple sources to efficiently manage network congestion [8].

This paper presents uvgComm 1.0, the first open-source P2P mesh video communication software that is implemented without WebRTC. Formerly known as Kvazzup [9], the software has been renamed to uvgComm in this work to emphasize its new video conferencing capabilities. uvgComm incorporates its own implementations of *Session Initiation Protocol* (SIP) [10] and *Session Description Protocol* (SDP) [11], of which the latter is extended for

P2P mesh configurations. It also improves connectivity and security by supporting IPv6 and ZRTP protocol. uvgComm P2P mesh network architecture ensures that only one encoder per sender is required in a multi-party communication. In addition, uvgComm utilizes the following open-source solutions:

- Kvazaar [12]: for video encoding;
- OpenHEVC [13]: for video decoding;
- opus [14]: for audio compression and decompression;
- libyuv [15]: for color space conversion;
- speex DSP [16]: for audio processing;
- uvgRTP [17]: for RTP packetization and delivery.

The rest of the paper is structured as follows. Section 2 gives an overview of the uvgComm architecture. Section 3 reports the performance results. Section 4 details the usage of uvgComm. Finally, Section 5 concludes the paper.

2 Overview of uvgComm Architecture

uvgComm is built using version 6 of Qt application framework. Figure 1 shows the high-level architecture of uvgComm, which is composed of three main modules: 1) *user interface (UI)*; 2) conference initiation & negotiation; and 3) media processing & delivery. There are also separate modules for system control.

2.1 User Interface

The uvgComm features a *graphical user interface (GUI)* that is designed for a seamless and intuitive user experience. Besides the main application windows, it offers a comprehensive settings menu that allows users to adjust uvgComm to their individual needs and preferences. The GUI also integrates a statistics window [18] that displays live performance metrics in easy-to-understand graphs, providing users with real-time insights into the QoE indicators of the system.

2.2 Conference Initiation & Negotiation

Conference initiation & negotiation module is made up of 1) SIP transport which converts transported strings into the uvgComm data structures and vice versa; 2) SIP transaction which maintains correct state within the SIP protocol throughout the communication session; and 3) SDP negotiation which negotiates the call parameters, such as how media are sent/received between participants.

SIP and SDP were chosen because of their widespread use, flexibility and proven reliability. uvgComm can utilize a SIP server, such as Kamailio [19], to form SIP connections. At start, uvgComm sends a REGISTER-request containing user and connection details to the server so that other SIP agents can call this user. All following INVITE-requests are routed via the SIP server to bypass firewalls. While not taking advantage of the advanced architecture that uvgComm offers, other applications that support SIP, SDP and a common set of media codecs could communicate with uvgComm even in P2P mesh mode.

Figure 2 illustrates the creation of new media connections when a new participant joins the session. The initial uvgComm

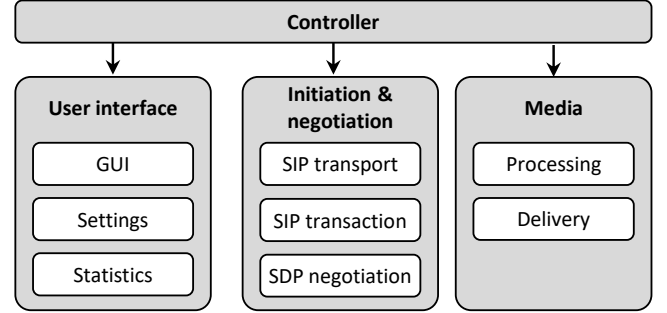


Figure 1: High-level architecture of uvgComm.

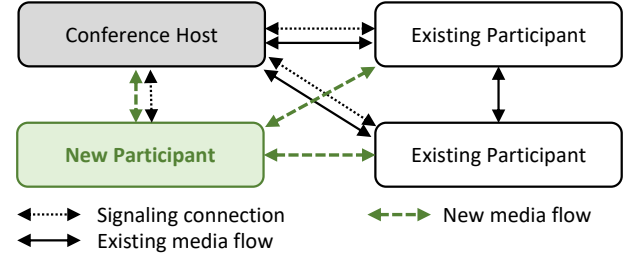


Figure 2: Additional media flows when a new participant joins the session.

Table 1: Application characterization with N conference participants

Software	Architecture	Encoders	ZRTP	OSS
Teams	SFU*	1	No	No
Zoom	SFU*	1	No	No
Google Meet	SFU*	1	No	No
Jitsi Meet	SFU	1	No	Yes
Mooz	P2P mesh	$N - 1$	No	Yes
uvgComm	P2P mesh	1	Yes	Yes

* Presumed closed-source architecture.

instance in the call acts as the host, utilizing SDP messages to manage the network topology and create the P2P mesh. It renegotiates the applied configuration with all participants each time a new participant joins, while existing media flows are not interrupted.

2.3 Media Processing & Delivery

uvgComm is built on open media components. The video workflow utilizes our Kvazaar *High efficiency Video Coding (HEVC)* encoder [10] for compressing raw video and the OpenHEVC decoder [11] for playback. The compatibility of video input is broadened with libyuv [13], which converts diverse camera formats into YUV 4:2:0 for Kvazaar. Audio is managed by Opus [12] codec. Audio enhancements like noise cancellation are carried out by SpeexDSP [14]. Media delivery is done by uvgRTP [15], which is our library for video packetization and transport. The transmitted media is encrypted using the *Secure Real-time Transport Protocol (SRTP)* and the keys for encryption are negotiated using ZRTP.

3 Performance Comparison

In our experiments, we evaluated the latency of P2P mesh-based uvgComm 1.0 and Mooz [4] as well as the SFU-based Microsoft Teams [20], Zoom [21], Google Meet [22], and Jitsi Meet [23]. Table 1 characterizes the benchmarked *open-source software* (OSS) and non-OSS applications. uvgComm is the only application that supports ZRTP and only P2P mesh-based solution that encodes all video streams with one encoder, enabling greater scalability. We were unable to setup Peer Calls in P2P mesh mode.

3.1 Experimental Setup

Figure 3 exemplifies the benchmarking setup for latency evaluation with two participants. The same setup was extended accordingly for more conference participants. All computers were provided a public IP address to not trigger NAT traversal in the software.

Each software was given a one-minute warm up time to minimize the effects of congestion control. On the sender end, Logitech C930 webcam was used to capture 120 Hz screen that showed a millisecond timer updated with each screen refresh. GoPro Hero 5 camera was set to take a benchmark photo every two seconds for 100 photos. The latency was obtained by subtracting the past timer value from the current timer value.

The benchmark photos were parsed with PaddlePaddleOCR [24] framework. The benchmark setup has inherent error from 1) sender Camera capture interval of 33 ms, 2) the timestamp display interval of 8.3 ms and 3) receiver display interval of 16.7 ms. This leads to maximum variability of 58 ms mitigated by capturing the timestamps 100 times and averaging results.

We opted not to measure image quality, bit rate, or CPU/GPU usage as these choices are mostly independent of the underlying conferencing architecture. For example, Mooz always sets its total bandwidth to 2 Mbit/s regardless of the participant count, which also affects CPU usage and making it difficult to draw conclusions.

These benchmarks are not meant to give an in-depth comparison between the evaluated software, but an idea of the benefits of P2P mesh-based architectures. On one hand, the computers were close to each another in the network, favoring the direct connections more than non-direct connections. On the other hand, the computers were close to the backbone network, providing a fast access to servers avoiding the worst case for server-based connections.

3.2 Experimental Results

Table 2 reports the latency measurement results in our test scenario. P2P mesh architectures perform better than those of SFU because of their ability to use direct connections. The only exception is Jitsi Meet which utilizes direct connections in two-participant calls [25]. With 3-5 participants, uvgComm can reach 25% lower latencies compared to average of 219 ms achieved by SFUs. The lower latency is a key factor in communication QoE and thereby the one of the main motivations for using P2P mesh architectures in multi-party video communication.

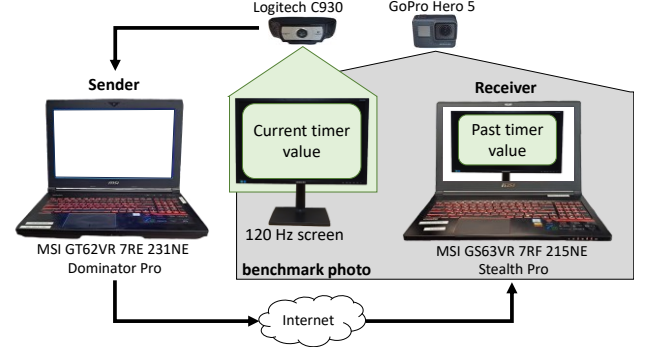


Figure 3: Benchmark test setup for latency evaluation.

Table 2: Latency results with 2 to 5 participants

Nb participants	2	3	4	5
Google Meet	255 ms	232 ms	216 ms	223 ms
Teams	217 ms	215 ms	226 ms	231 ms
Zoom	209 ms	213 ms	207 ms	211 ms
Jitsi Meet	142 ms	220 ms	218 ms	218 ms
Mooz	159 ms	158 ms	152 ms	155 ms
uvgComm	146 ms	166 ms	165 ms	164 ms

4 Software Deployment

Here we will present the compilation, user guide and intended audience of uvgComm. The source code of uvgComm is available on GitHub at:

<https://github.com/ultravideo/uvgComm>

along with detailed build instructions.

4.1 Compilation

uvgComm is compatible with both Linux and Windows. The software utilizes CMake for its build process to automatically download and/or compile any missing dependencies. One can refer to the `/BUILDING.md` file in the repository for detailed build instructions of uvgComm.

4.2 User Guide

The deployment of uvgComm in multi-participant video communication requires a computer with a camera and microphone. Additionally, an account on a SIP server such as open-source Kamailio [19] is necessary to bypass firewalls. uvgComm repository includes setup instructions for Kamailio in `/kamailio/README.md`.

uvgComm supports both direct SIP calling between participants and calling via a SIP proxy server, but the usage of a SIP proxy is recommended for the sake of security and reliability. If a SIP account is applied, users must input their username and

password when first launching uvgComm. They will also be prompted to enable *Session Traversal Utilities (STUN)* connections, which facilitate connections through most types on NATs. Currently, uvgComm does not support calling from asymmetric NATs utilized in IPv4 based mobile networks and large office wireless networks, but this will be addressed in the future.

After initial setup, users can add new contacts to their contact list and initiate calls. In case of SIP errors, the user is notified with a pop-up window. uvgComm provides detailed options for configuring the Kvazaar encoder and enables users to view the QoE statistics for ongoing sessions.

4.2 Intended Audience

uvgComm is ideally suited for scenarios, where the deployment of webRTC is impractical or too restrictive. It offers a low-latency, high-privacy, open-source alternative to commercial video communication solutions.

Additionally, uvgComm is a well-suited framework for testing novel technologies within the realm of real-time communication. It enables researchers and developers to experiment with and evaluate the impact of various encoding tools and configuration on energy consumption, latency, or QoE. Thanks to its permissive ISC license and adaptable framework, uvgComm serves as a valuable resource for both academic research and industrial applications.

5 Conclusion

This paper introduced uvgComm that is the first open-source P2P mesh video communication application implemented without WebRTC. Lack of WebRTC has enabled a processing architecture with only one video encoder per sender to run all media flows and support for ZRTP encryption key exchange protocol. The permissive ISC license allows its use in both industrial and academic applications. By adopting P2P mesh architecture, uvgComm can reach 25% lower latencies in multi-party conference compared to SFU architecture-based applications.

In the future, we plan to integrate coupled congestion control algorithm to uvgComm and investigate methods for improved bandwidth allocation in multi-party P2P mesh-based conferences.

Acknowledgements

This work was supported in part by the *AI-based Situational Awareness (AISA)* project led by Nokia and funded by Business Finland, and the Research Council of Finland (decision no. 349216).

References

- [1] J. He, M. Ammar, E. Zegura, E. Halepovic, and T. Karagioules, "QoE metrics for interactivity in video conferencing applications: definition and evaluation methodology," in *Proc. ACM Multimedia Syst. Conf.*, Bari, Italy, Apr. 2024.
- [2] "ITU-T recommendation G.114. one-way transmission time," ITU-T, Geneva, May 2003.
- [3] J. Steiner, "Peer Calls," rondomoon, [Online]. Available: <https://peercalls.com/>.
- [4] "Mooz," [Online]. Available: <https://mooz.muzam1l.com/>.
- [5] "WebRTC main repository," Google, [Online]. Available: https://webrtc.googlesource.com/src/+refs/heads/main/api/video_codec/video_encoder/software_fallback_wrapper.h.
- [6] P. Zimmermann, A. Johnston, and J. Callas, "IETF RFC 6189 ZRTP: media path key agreement for unicast secure RTP," IETF, Apr. 2011, [Online].
- [7] S. Holmer, H. Lundin, G. Carlucci, L. De Cicco, and S. Mascolo, "IETF RFC draft: a google congestion control algorithm for real-time communication," Jul. 2016, [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-rmcat-gcc-02>.
- [8] V. Singh, A. A. Lozano, and J. Ott, "Performance analysis of receive-side real-time congestion control for WebRTC," in *Proc. Int. Packet Video Workshop*, San Jose, CA, USA, Dec. 2013.
- [9] J. Räsänen, M. Viitanen, J. Vanne, and T. D. Hämäläinen, "Kvazzup: open software for HEVC video calls," in *Proc. Int. Symp. Multimedia*, Taichung, Taiwan, Dec., 2017.
- [10] J. Rosenberg *et al.*, "IETF RFC 3261 SIP: session initiation protocol," IETF, Jun. 2002, [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3261>.
- [11] Jacobson and Perkins, "IETF RFC 8866 SDP: session description protocol," IETF, Jan. 2021, [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8866>.
- [12] A. Lemmetti, M. Viitanen, A. Mercat, and J. Vanne, "Kvazaar 2.0: fast and efficient open-source HEVC inter encoder," in *Proc. ACM Multimedia Syst.*, Istanbul, Turkey, May 2020.
- [13] W. Hamidouche, M. Raulet, and O. Deforges, "Real time shvc decoder: implementation and complexity analysis," in *Proc. Int. Conf. Image Process.*, Paris, France, Oct. 2014.
- [14] "Opus interactive audio codec," Xiph.Org Foundation and IETF, [Online]. Available: <https://opus-codec.org/>.
- [15] "libyuv," Google, [Online]. Available: <https://chromium.googlesource.com/libyuv/libyuv/>.
- [16] "Speex: a free codec for free speech," Xiph.Org, [Online]. Available: <https://www.speex.org/>.
- [17] H. Tampio, J. Räsänen, M. Viitanen, A. Mercat, G. Gautier, and J. Vanne, "uvgRTP 3.0: towards V3C volumetric video communication," in *Proc. ACM Multimedia Syst. Conf.*, Bari, Italy, Apr. 2024.
- [18] J. Räsänen, M. Viitanen, J. Vanne, and T. D. Hämäläinen, "Live demonstration: Kvazzup 4K HEVC video call," in *Proc. IEEE Int. Symp. Multimedia*, Taichung, Taiwan, Dec. 2018.
- [19] "Kamailio SIP server," The Kamailio SIP Server Project, [Online]. Available: <https://www.kamailio.org/>.
- [20] "Microsoft Teams," Microsoft Corporation, [Online]. Available: <https://www.microsoft.com/en-us/microsoft-teams/group-chat-software>.
- [21] "Zoom Meetings," Zoom Video Communications, Inc., [Online]. Available: <https://www.zoom.com/en/products/virtual-meetings/>.
- [22] "Google Meet," Google LLC, [Online]. Available: <https://workspace.google.com/products/meet/>.
- [23] B. Grozev, L. Marinov, V. Singh, and E. Iovv, "Last N: relevance-based selectivity for forwarding video in multimedia conferences," in *Proc. Multimedia Syst. Conf.*, Portland, OR, USA, Mar. 2015.
- [24] "Paddle Paddle-Parallel distributed deep learning, efficient and extensible deeplearning framework," PaddlePaddle developers, [Online]. Available: <https://www.paddlepaddle.org.cn/en>.
- [25] G. Politis, B. Grozev, P. Domas, E. Iovv, and T. Noël, "Experimental evaluation of dynamic switching between one-on-one and group video calling," in *Proc. Principles, Syst. Applications of IP Telecomm.*, Chicago, IL, USA, Oct. 2018.